

Particle advection simulation

Viacheslav Kruglov

July 17, 2024

Abstract

—

1 Introduction

2 Description

Calculations consist of two parts:

1. preparation of the vector fields, which includes reading the data files and reconstruction of the smooth fields in space and time,
2. simulation of advection of particles driven by the reconstructed fields. Particles are propagated with explicit Euler method.

3 Interpolation of the spatial field

We get the velocity fields from HYCOM (for ocean currents) and from Copernicus (for winds). The data is stored as multidimensional tables in netCDF format. We will not discuss the tedious process of reading the data files. After the preprocessing of the data we get rectangular grids of eastward and northward components of velocity fields: $u_j(t)$ and $v_j(t)$. The coordinates are degrees of latitude φ and longitude λ . Here j is the spatial index of the grid point. The actual sorting does not matter, because we build the kd-tree for nearest neighbor search. In principle the grid points can be scattered nonregularly. Importantly the positions of grid points must be constant in

time. We omit grid points that lie on land to speed up the simulation and to save memory. t represents time, it is discrete in the data, but we interpolate it with cubic splines to get continuous dependence on time.

Our goal is simulation on very large scales. The fact that the surface of the Earth is not flat can not be overlooked, any flat projection brings irremovable distortions of distances, angles, areas of shapes or all of it together. We utilize the obvious fact that the surface of the Earth is a smooth manifold and calculate flat projections locally for short-distance advection. With that we aim to minimize distortions.

Let's call the position of the particle a query point. At a given time t we choose the query point (λ, φ) to be an origin of a local azimuthal equidistant projection. This projection does not distort the distances, which are calculated as Great Circle Distances (in radians):

$$s_j = \text{atan} \frac{|\bar{n} \times \bar{n}_j|}{\langle \bar{n} \cdot \bar{n}_j \rangle} = \text{atan} \frac{\sqrt{1 - (\sin \varphi \sin \varphi_j + \cos \varphi \cos \varphi_j \cos(\lambda - \lambda_j))^2}}{\sin \varphi \sin \varphi_j + \cos \varphi \cos \varphi_j \cos(\lambda - \lambda_j)}, \quad (1)$$

where s_j is the Great Circle Distance between the query point (λ, φ) and a grid point j with lat-lon coordinates (λ_j, φ_j) . $\bar{n}$ and $\bar{n}_j$ are normal unit vectors at these points, $|\bar{n} \times \bar{n}_j|$ and $\langle \bar{n} \cdot \bar{n}_j \rangle$ are the sine and cosine of the s_j , that gives you an idea how to derive the equation.

We interpolate field values between grid points by calculating weighted average of the values at the nearest grid points and weight functions decrease with the distance from the query point. Nearest neighbors are searched on a kd-tree built at the start of the simulation. The radius of search is chosen in such a way ($R \approx 1.5L$, where L is a grid step) that in any typical position of a query point there are at least 4 grid points inside the search spot.

The weight function is:

$$\mu(s_j) = \exp\left(-\frac{s_j^2}{L^2}\right), \quad (2)$$

where s_j is the distance from the query point to the grid point with index j , L^2 governs the shape of the Gaussian. It is found out by trial and error that the shape must be related to the grid step L for better approximation. Obviously if the query point coincides with the grid point, than the weight is 1. Weights of grid points further away decrease smoothly with the distance.

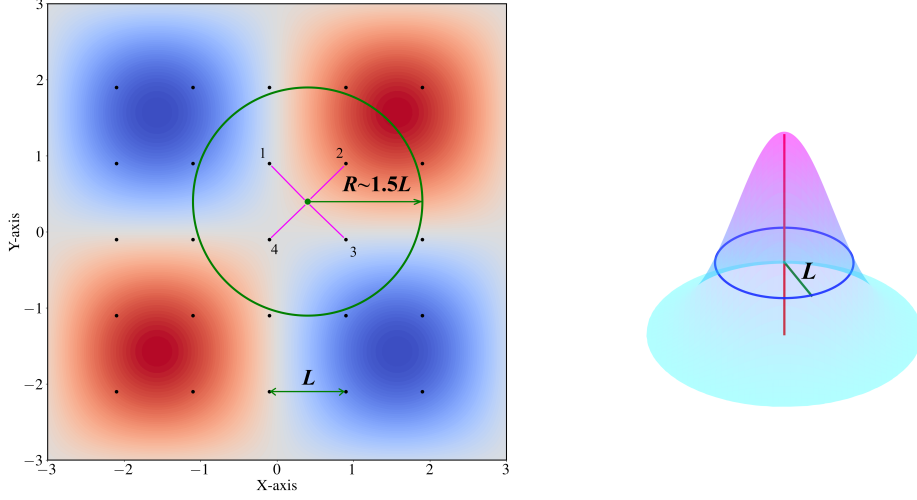


Figure 1: Left panel: Grid and a radius of search of nearest grid points. Right panel: Gaussian weight function

The measure of the grid point j in the approximation is then

$$\rho_j = \frac{\mu(s_j)}{\sum_{s < R} \mu(s)}, \quad (3)$$

where the sum in the denominator is carried for every grid point in the search radius.

The interpolated values of the eastward and northward components of the vector field are then

$$u(\lambda, \varphi, t), v(\lambda, \varphi, t) = \sum_{s_j < R} \rho_j u_j(t), \sum_{s_j < R} \rho_j v_j(t) \quad (4)$$